VEHICLE INSURANCE MANAGEMENT SYSTEM

A Complete Guide to Workflows, Security and Systems Engineering

Document Type:	Workflow Reference, LocalStorage Schemas & Code Documentation
Created Date:	July 12, 2026
Target Audience:	Developers, Quality Assurance & Systems Engineering Teams
Architecture Pattern:	Client-side Single Page Architecture (Vanilla HTML5 / CSS3 / ES6 JS)
Primary Database:	Web HTML5 LocalStorage Sandbox

1. System Workflow & Data Operations

The Vehicle Insurance Management System (VIMS) operates as a client-side architecture where the client browser serves as both the processing node and database engine. Key system benefits include: zero-latency rendering, offline compatibility, and sandbox containment. Let us explore the system's operational lifecycle:

1.1 User Roles & Transaction Flow

The system enforces a multi-role model divided between **Administrator** and **Underwriter** profiles. Data access is segmented via primary-foreign relationships tracked inside LocalStorage variables.

Role Name	Target Module / Folder	Key Responsibilities & Operational Actions
Administrator	/Group_project/admin/	• Manages the lifecycle of underwriter accounts. • Registers new underwriter profiles with names, emails, and passwords. • Resets or updates active credentials. • Deactivates underwriter records to revoke database query access.
Underwriter	/Group_project/underwriter/	• Manages customer vehicle policies. • Enters policy metrics (vehicle, customer name, chassis/engine code). • Transitions policy types and handles annual premium calculation. • Renews expiring customer records, adding a 365-day extension.

2. User Authentication & Authorization Engine

The security subsystem evaluates user roles during sign-in and populates active session identifiers. The Administrator credentials are statically evaluated to protect global operations, whereas Underwriter accounts are query-matched against registered entries in LocalStorage.

```
// File: Group_project/components/login/login.js
// Handles verification of Administrator & Underwriter roles
if (role === "Administrator") {
    if (username === "admin" && password === "admin123") {
        handleLoginSuccess("../../admin/admin_dashboard/admin_dashboard.html", "Administrator");
    } else {
        handleLoginFailure("Invalid Administrator credentials. Please verify credentials.");
    }
} else if (role === "Underwriter") {
    const underwriters = JSON.parse(localStorage.getItem("underwriters")) || [];
    const underwriter = underwriters.find(u => u.underwriterId === username && u.password === password);
    if (underwriter) {
        localStorage.setItem("loggedInUnderwriter", JSON.stringify(underwriter));
        handleLoginSuccess("../../underwriter/underwriter_dashboard/underwriter_dashboard.html", "Underwrit
er");
    } else {
        handleLoginFailure("Invalid Underwriter ID or Password.");
    }
}
```

When handleLoginSuccess runs, it sets key session states in localStorage, including: isLoggedIn = true, loggedInRole, and loggedInUser (containing the active ID and status to guard page views).

3. Underwriter Business Logic & Policy Lifecycle

3.1 Automatic Premium Calculations

Premium values are automatically generated inside the form using a strict cross-reference between vehicle type (2 Wheeler vs. 4 Wheeler) and the chosen coverage class (Third Party vs. Full Insurance).

Vehicle Type	Third Party Insurance Premium	Full Insurance Premium
2 Wheeler	■ 2,500	■ 4,500
4 Wheeler	■ 6,000	■ 10,000

3.2 Policy Creation & Expiry Configurations

To establish high policy standard compliance, the system restricts dates securely:

- **From Date:** Implemented as a calendar selector allowing selection of today or any future date.
- **To Date:** Made strictly read-only and automatically locked to exactly 1 year (365 days) from the selected From Date.
- **Form validation:** The system enforces calendar validation rules at runtime, preventing chronological conflicts.

```
// File: Group_project/underwriter/create_vehicle_insurance/create_vehicle_insurance.js
// Calculates exact 365 days offset for the locked read-only To Date
fromDate.addEventListener("change", function () {
    if (fromDate.value !== "") {
        let date = new Date(fromDate.value);
        date.setDate(date.getDate() + 365);
        let day = String(date.getDate()).padStart(2, "0");
        let month = String(date.getMonth() + 1).padStart(2, "0");
        let year = date.getFullYear();
        toDate.value = `${day}-${month}-${year}`; // DD-MM-YYYY format
    } else {
        toDate.value = "";
    }
});
```

By outputting the calculated toDate in the read-only format as DD-MM-YYYY directly, the application ensures that the display layers and lists operate seamlessly without complex conversion loops.

3.3 Expiry Logic & Active Status Checks

On policy listing pages, JavaScript evaluates whether a policy is active by splitting the stored expiry date string and checking it against the current system date.

```
// File: Group_project/underwriter/view_policy/view_policy.js
// Determines if the policy is active or expired relative to today
let status = "Expired";
if (policy.toDate) {
    const parts = policy.toDate.split("-");
    const expiryDate = new Date(parts[2], parts[1] - 1, parts[0]);
    expiryDate.setHours(0, 0, 0, 0);
    if (expiryDate >= today) {
        status = "Active";
    }
}
```

3.4 Policy Updates & Dynamic Change Logs

Policy updates are governed by strict business constraints. An underwriter is permitted to downgrade coverage from **Full Insurance** to **3rd Party Insurance** (which lowers the premium to ■2,500), but upgrading from Third Party to Full Insurance is restricted. The transaction stores changes and updates the layout dynamically using an internal tracking schema.

```
// File: Group_project/underwriter/update_policy/update_policy.js
// Updates policy fields, saves to LocalStorage, and renders the dynamic status log
policies[policyIndex].insuranceType = "3rd Party Insurance";
policies[policyIndex].premium = "2500";
policies[policyIndex].isUpdated = true;
policies[policyIndex].updatedAt = new Date().toLocaleString();
localStorage.setItem("policies", JSON.stringify(policies));
displayUpdateStatus(policies[policyIndex]);
```

4. Responsive Grid, Sidebar & Overflow Calibrations

During interface audits, horizontal page scrollbars were identified on desktop views due to spacing conflicts between fixed element margins and standard container calc widths. These were corrected by aligning element coordinates to standard sidebar boundaries.

Component Element	Old Conflicting Rule	New Corrected Rule	Operational Result
.sidebar-frame	width: 330px;	width: 330px;	Maintained as the sidebar baseline anchor.
.container / .content-wrapper	margin-left: 300px; width: calc(100% - 260px);	margin-left: 330px; width: calc(100% - 330px);	Offsets clear the sidebar perfectly and container width fits exactly, resolving the right-side horizontal page overflow.
.navbar-frame	left: 280px; width: calc(100% - 280px);	left: 330px; width: calc(100% - 330px);	Corrects navbar alignment starting from the right-hand boundary of the sidebar.

5. Summary of Architectural Integration

By consolidating operational security, input constraints, and layout calculations directly inside optimized browser modules, VIMS provides a reliable client-side enterprise environment. The relational links tracked in the LocalStorage database preserve structural logic, making the overall workflow stable and maintainable for future enhancements.